

– ORIGINAL ARTICLE –

Delta BPTT: Event-Gated Backpropagation Through Time for Efficient Spiking Sequence Representations

Delta BPTT: Retropropagación a Través del Tiempo Regulada por Eventos para Representaciones Eficientes de Secuencias con Impulsos Neurales

Muhammad Akhyar¹ 

¹Independent Researcher
akhyarsafrudin@gmail.com

Abstract

Spiking Neural Networks (SNNs) offer an event-driven alternative to conventional deep networks by replacing dense floating-point multiplications with sparse accumulations. When SNNs are trained with Backpropagation Through Time (BPTT), however, parameter-gradient accumulation traditionally retains dense Multiply-Accumulate (MAC) operations. In this paper, we present **Delta BPTT**, an event-gated execution formulation of BPTT for binary spike representations in which input spikes gate weight-vector accumulation, turning both forward recurrent projections and weight-gradient updates into conditional vector additions. Evaluated on a 65K-trainable-parameter recurrent pooler attached to a frozen static embedding table across the ALL-STS teacher-fidelity aggregate and STS-B benchmarks (29,720 sentence pairs), Delta BPTT eliminates spike-gated MACs (67.2% activity sparsity matching the forward pass) and accelerates CPU training by $2.44\times$ over Standard BPTT with a gradient cosine similarity of 0.9982. Furthermore, the recurrent embedder architecture reduces CPU training time by $4.13\times$ compared to Spike-Overlap attention while maintaining high correlation fidelity. An operation-based 45 nm energy proxy suggests a whole-pooler energy reduction of $\approx 2.7\times$ (up to $15.6\times$ for the isolated spike-gated weight update).

Index Terms—Spiking Neural Networks, Backpropagation Through Time, Sequence Representation, Sentence Embeddings, Neuromorphic Computing, Delta BPTT.

1 INTRODUCTION

The computational overhead of dense sentence embedders [1], such as SBERT [2], is dominated by the matrix multiplications of scaled dot-product attention [3]. While numerous efficient Transformer variants mitigate this quadratic $\mathcal{O}(L^2)$ cost [4], they continue to rely on continuous floating-point arithmetic. Spiking Neural Networks (SNNs) take a different route: because their activations are binary spike events, synaptic integration can in principle be executed as sparse, event-driven accumulation rather than dense multiplication, which makes SNNs attractive for neuromorphic hardware [5]. Recent neuromorphic NLP research has ported self-attention into the spiking domain [6], [7], and frameworks such as SpikingJelly [8] have accelerated the validation of surrogate

Resumen

Las Redes Neuronales de Impulsos (SNNs) ofrecen una alternativa por eventos frente a las redes profundas al sustituir multiplicaciones en coma flotante por acumulaciones dispersas. Sin embargo, el entrenamiento con Retropropagación a Través del Tiempo (BPTT) mantiene operaciones densas de Multiplicación y Acumulación (MAC) durante la acumulación de gradientes. En este artículo presentamos **Delta BPTT**, una formulación de ejecución regulada por eventos para impulsos binarios donde los impulsos de entrada controlan la acumulación de vectores de pesos, convirtiendo las proyecciones y actualizaciones de gradientes en sumas vectoriales condicionales. Evaluado en un agrupador recurrente de 65K parámetros entrenables acoplado a una tabla de embebidos estáticos congelados en las pruebas ALL-STS y STS-B (29.720 pares de oraciones), Delta BPTT elimina las operaciones MAC reguladas por impulsos (67,2% de dispersión) y acelera el entrenamiento en CPU en $2,44\times$ frente a BPTT Estándar con una similitud coseno de gradientes de 0,9982. Asimismo, la arquitectura del agrupador recurrente reduce el tiempo de entrenamiento en $4,13\times$ respecto a la atención Spike-Overlap. Una estimación energética a 45 nm sugiere una reducción de energía del agrupador completo de $\approx 2,7\times$ (hasta $15,6\times$ para el término de actualización de pesos regulado por impulsos).

Palabras clave—Redes Neuronales de Impulsos, Retropropagación a Través del Tiempo, Representación de Secuencias, Embebidos de Oraciones, Computación Neuromórfica, Delta BPTT.

gradient training in deep spiking architectures.

A frequently overlooked bottleneck, however, lies not in the *forward* pass—where spike-driven sparsity is well established—but in the *backward* pass. Standard BPTT with surrogate gradients [9] propagates continuous error signals backward through the unrolled network, and the weight-gradient accumulation $\Delta \mathbf{W} = \sum_t \delta^{(t)} (\mathbf{X}^{(t)})^T$ is conventionally evaluated as a dense outer product even when $\mathbf{X}^{(t)}$ is a binary spike vector. This is mathematically unnecessary: when the gating signal is strictly binary, the outer product degenerates into a selection of rows, and selection can be executed as conditional accumulation. Existing SNN training formulations do not expose this structure as an explicit event-driven execution rule with a precise operation accounting. This raises two questions:

(1) *Can the spike-gated phases of BPTT—the forward projection and the weight-gradient accumulation—be executed as conditional additions, and what exactly is saved?* (2) *Are explicit spatial attention mechanisms necessary for sentence-level representation, or does native temporal recurrence paired with such a learning rule suffice?*

We address both questions with **Delta BPTT**, an event-gated recurrent training rule. The central observation is algebraic: for a binary vector $\mathbf{S}^{(t)} \in \{0, 1\}^{D_{in}}$, the identity $\mathbf{W}^T \mathbf{S}^{(t)} = \sum_{d: S_d^{(t)}=1} \mathbf{W}_d$ is exact, and the right-hand side contains no multiplication. The same identity applies to the gradient outer product $\sum_t \mathbf{S}^{(t)} (\delta^{(t)})^T$. Delta BPTT executes both phases by iterating over active spike events only. We emphasize the scope of this claim throughout: multiplications inside the neuron dynamics, the temporal error recurrence, the surrogate-gradient mask, and the error back-projection to earlier layers are *not* removed. Our empirical findings indicate that the recurrent pooler trained with this rule is sufficient for encoding semantic structure in the sentence-embedding setting, reaching 93.5% of the STS-B Spearman correlation of SpikingBERT [10] with $770\times$ fewer parameters and no GPU.

1.1 Contributions

- **Event-Gated Execution Rule.** We introduce Delta BPTT, an event-gated execution rule for binary spike representations that converts both forward recurrent weight projections and backward weight-gradient updates into conditional vector additions, bypassing dense outer products without modifying the underlying learning objective.
- **Exact Algebraic Equivalence & Operation Accounting.** We prove that Delta BPTT is algebraically equivalent to Standard BPTT in exact arithmetic ($\Theta(LD_{in}D_{out})$ complexity with active work $aLD_{in}D_{out}$ ACs at spike activity rate a), and delimit precisely which operations are converted (spike-gated MACs) versus retained (neuron dynamics, error back-projection).
- **Empirical Validation on Sequence Representations.** We validate Delta BPTT on semantic sentence embedding benchmarks (STS-12–16, STS-B, SICK-R), demonstrating that our 65K trainable parameter pooler recovers 88.3% of the teacher’s zero-shot performance, yields a $2.44\times$ CPU training speedup over dense BPTT ($4.13\times$ over attention baselines), and aligns with Standard BPTT gradients ($\cos = 0.9982$, relative error 0.34%).

2 RELATED WORK

2.1 Spiking Neural Networks for Language

Recent work has ported Transformer primitives into the spiking domain. SpikFormer [11] introduced spike-driven self-attention for vision, and SpikeGPT [7] scaled spiking architectures to generative language modeling with up to 216M parameters. For encoder-style representations, SpikeBERT [12] applies two-stage knowledge distillation

from BERT, while SpikingBERT [10] integrates spiking neurons with BERT-style pretraining and reports GLUE results—including a Spearman correlation of 0.819 on STS-B—with approximately 50M parameters; Spikingformer [13] stabilizes spike-driven residual learning and reports 0.545 on STS-B with 66M parameters. Two observations motivate our study. First, these models retain an explicit $\mathcal{O}(L^2)$ attention mechanism that dominates their operation counts. Second, none of them targets *sentence embeddings* or reports the standard STS suite (STS-12 through STS-16, STS-B, SICK-R) against human judgments; to the best of our knowledge, our 65K-parameter recurrent pooler is among the first SNNs evaluated directly on this suite, though we do not claim absolute priority.

2.2 Efficient Sentence Embedding

Dense sentence embedders are typically obtained by fine-tuning pretrained encoders with siamese [2] or contrastive objectives [14], and compressed via knowledge distillation [15]–[17] or quantization. These models define the accuracy frontier—DistilBERT reports 0.869 Spearman on STS-B [16]—but rely on continuous floating-point matrix multiplications. Static embeddings with mean pooling (Word2Vec, GloVe) remain competitive low-cost baselines and are included in our evaluation. The standard evaluation convention for this task family is correlation against human gold labels [18], [19], which we adopt for the zero-shot track of our evaluation.

2.3 Temporal Credit Assignment and Event-Driven Training

Surrogate-gradient BPTT [9] is the dominant training method for deep SNNs, but its backward pass is dense in time and in space. Alternatives include truncated BPTT, eligibility-propagation rules (e-prop) [20], event-driven gradient methods for continuous-time spiking dynamics (e.g., EventProp) [21], and online approximations such as real-time recurrent learning; sparse matrix–vector kernels form the classical computing counterpart of spike-gated accumulation [22]. Delta BPTT is orthogonal to these methods: it does not approximate or truncate the temporal credit assignment—the recurrence is identical to Standard BPTT—but changes the *execution* of the spike-gated operations from dense multiplication to conditional accumulation. To our knowledge, existing SNN training pipelines do not state this execution rule and its operation accounting explicitly.

2.4 Energy Accounting for SNNs

SNN efficiency claims conventionally rely on operation counting: spike-driven accumulate operations (SOPs/ACs) are contrasted with multiply–accumulate operations (MACs), weighted by the 45 nm energy figures of Horowitz [23] (0.9 pJ per 32-bit accumulation versus 4.6 pJ per 32-bit MAC) [8], [11]. This proxy omits memory access and data movement, which can dominate real energy budgets. Moreover, existing analyses focus on the *forward*

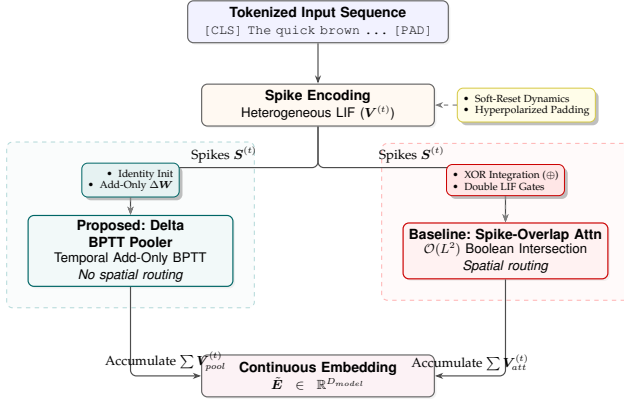


Fig. 1. Architectural flowchart contrasting the proposed Delta BPTT Recurrent Pooler (core contribution) against the $\mathcal{O}(L^2)$ Spike-Overlap Attention baseline (secondary, ablation-only component).

TABLE 1
Notation used throughout the paper.

Symbol	Meaning
$L (=T)$	sequence length in tokens = discrete timesteps
D_{in}, D_{out}	pooler input / output dimension
D	shorthand when $D_{in} = D_{out} = D$
$a \in [0, 1]$	spike <i>activity</i> rate (fraction of entries = 1)
$s = 1 - a$	spike <i>sparsity</i> ($s \approx 0.672$ measured, $a \approx 0.328$)
$\mathbf{S}^{(t)} \in \{0, 1\}^{D_{in}}$	embedding-layer spike vector at step t
$\mathbf{V}^{(t)}, \mathbf{U}^{(t)}$	membrane potential, pre-reset potential
β, θ	per-neuron leak decay and threshold (heterogeneous)
$\mathbf{W} \in \mathbb{R}^{D_{in} \times D_{out}}$	pooler weight matrix; $\mathbf{W}_{d,:}$: its d -th row
$\delta^{(t)} \in \mathbb{R}^{D_{out}}$	masked temporal error at step t
$\sigma'(\cdot)$	rectangular surrogate-gradient mask (window γ)
$\mathbf{E}, \bar{\mathbf{E}}$	sentence embedding and its ℓ_2 normalization

pass; the backward pass of BPTT is tacitly assumed to require dense MACs throughout. We therefore report the theoretical proxy and measured CPU latency side by side and treat the proxy strictly as an upper-bound estimate, never as a hardware measurement.

3 METHODOLOGY: DELTA BPTT AND ARCHITECTURAL DYNAMICS

Our spiking sentence embedder constructs continuous semantic representations from discrete spike events. The pipeline consists of spike encoding, a sequence aggregation mechanism (the proposed recurrent pooler, or the attention baseline), and knowledge-distillation training. Table 1 fixes the notation used throughout.

3.1 Spike Encoding and Heterogeneous Dynamics

Raw text is tokenized (BPE; vocabulary file loaded at runtime) into indices $\mathbf{x}^{(1:L)}$. A spiking embedding layer maps each token to a temporal spike train; the temporal dimension coincides with the sequence dimension, i.e., token position t is timestep t . Writing $\mathbf{W}_{emb}[\mathbf{x}^{(t)}] \in \mathbb{R}^{D_{in}}$ for the embedding row of token $\mathbf{x}^{(t)}$ (equivalently, a matrix product with a one-hot vector), the membrane dynamics

follow a Leaky Integrate-and-Fire (LIF) model with soft reset:

$$\mathbf{U}_{emb}^{(t)} = \min(1.0, \beta_{emb} \odot \mathbf{V}_{emb}^{(t-1)} + \mathbf{W}_{emb}[\mathbf{x}^{(t)}]), \quad (1)$$

$$\mathbf{S}_{emb}^{(t)} = H(\mathbf{U}_{emb}^{(t)} - \theta_{emb}) \in \{0, 1\}^{D_{in}}, \quad (2)$$

$$\mathbf{V}_{emb}^{(t)} = \mathbf{U}_{emb}^{(t)} - \mathbf{S}_{emb}^{(t)} \odot \theta_{emb}, \quad (3)$$

where $H(\cdot)$ is the element-wise Heaviside step function and the $\min(1.0, \cdot)$ clamp bounds the pre-reset potential. We use a *soft reset* (subtracting the threshold) rather than a hard reset to zero, preserving fractional sub-threshold residue across time. Leak decays and thresholds are heterogeneous across dimensions: $\beta_i = 1 - 2^{-k_i}$ with k_i sampled uniformly from $\{2, \dots, 7\}$ (i.e., $\beta_i \in [0.75, 0.9922]$), forcing multi-timescale temporal processing at ingestion. These element-wise operations ($\beta \odot \mathbf{V}$, the clamp, the comparison, the reset subtraction) are part of the computation that Delta BPTT does *not* remove; they appear in both the Standard and Delta variants.

Sequence Padding and Hyperpolarization. To handle variable sequence lengths without attention masks, the embedding row of the $\langle \text{PAD} \rangle$ token (index 0) is overridden to -1.0 in every dimension. Padding steps therefore inject a strongly negative current, hyperpolarizing the membrane ($U < 0$), which guarantees that padding emits exactly zero spikes and contributes no noise to temporal pooling.

3.2 Ablation Baseline: Spike-Driven Spike-Overlap Attention

To isolate the contribution of spatial routing, we formalize a spike-driven self-attention mechanism used *only* as an ablation baseline; it is not part of the Delta BPTT contribution. Embedding spikes $\mathbf{S}_{emb}$ are projected into Query, Key, and Value spike trains by independent LIF layers. For query token i and key token j , the raw affinity is a Boolean match count:

$$M_{i,j}^{(t)} = \sum_{d=1}^{D_{model}} (Q_{i,d}^{(t)} \wedge K_{j,d}^{(t)}), \quad (4)$$

normalized by $\max_k (M_{i,k}^{(t)} + \epsilon)$. The normalized scores drive an auxiliary LIF layer that emits attention-mask spikes $\mathbf{S}_{scores}$, which gate the Value spikes by accumulation. Contextual features are then combined with the original embedding spikes through an exclusive-OR:

$$\mathbf{S}_{agg}^{(t)} = \mathbf{S}_{emb}^{(t)} \oplus \mathbf{S}_{att}^{(t)}. \quad (5)$$

The pairwise match counts cost $\mathcal{O}(L^2 \cdot D)$ per sequence (plus $\mathcal{O}(L \cdot D^2)$ for the Q/K/V projections), which is precisely the quadratic routing cost the recurrent pooler avoids.

To isolate the benefit of temporal pooling from the cost of spatial attention, we construct a Spike-Driven Spike-Overlap Attention baseline. For query $\mathbf{Q}^{(t)}$, key $\mathbf{K}^{(t')}$, and value $\mathbf{V}^{(t')}$ spike trains, the affinity matrix is computed

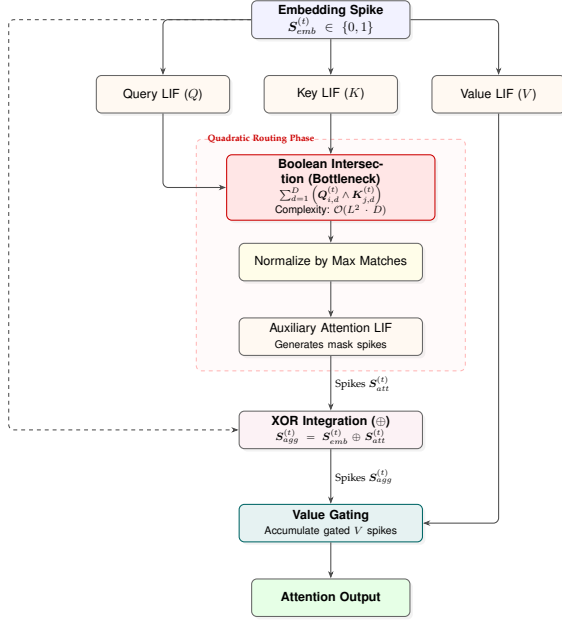


Fig. 2. Detailed computational flow of the Spike-Overlap Attention baseline at a single timestep (ablation-only component).

via bitwise AND (spike overlap) rather than floating-point dot products:

$$A_{t,t'} = \sum_{d=1}^D Q_d^{(t)} \wedge K_d^{(t')}. \quad (6)$$

The attention map is normalized via softmax and applied to $V^{(t')}$. Crucially, storing and contracting the $L \times L$ affinity matrix A requires $\mathcal{O}(L^2D)$ operations per sequence and $\mathcal{O}(L^2)$ memory per head, creating a spatial routing bottleneck that scales quadratically with sequence length L .

3.3 The Delta BPTT Execution Rule

We now present the core contribution: an event-gated execution formulation that eliminates Multiply-Accumulate (MAC) operations from the two spike-gated phases of BPTT.

Mathematical foundation. Consider a dense weight matrix $W \in \mathbb{R}^{D_{in} \times D_{out}}$ and a binary spike vector $S \in \{0,1\}^{D_{in}}$. The matrix-vector product $W^T S$ evaluates as:

Proposition 1 (Spike-Gated Row Accumulation). *For $W \in \mathbb{R}^{D_{in} \times D_{out}}$ and $S \in \{0,1\}^{D_{in}}$,*

$$W^T S = \sum_{d=1}^{D_{in}} W_{d,:} S_d = \sum_{d: S_d=1} W_{d,:}, \quad (7)$$

where the last expression contains no multiplication. The identity is exact in real arithmetic; in floating point it differs from dense evaluation only by the order of summation.

The forward projection is therefore executed by iterating over active spikes:

$$X_{pool}^{(t)} = \sum_{d=1}^{D_{in}} W_{d,:} \cdot \mathbb{I}[S_{emb,d}^{(t)} = 1], \quad (8)$$

followed by the pooler LIF update (Eqs. 1–3 with pooler parameters). The continuous sentence embedding is the ℓ_2 -normalized temporal sum of post-reset membrane potentials:

$$E = \sum_{t=1}^T V_{pool}^{(t)}, \quad \tilde{E} = \frac{E}{\|E\|_2}. \quad (9)$$

Learning via Delta BPTT. Let $\delta_{out}^{(t)} = \partial \mathcal{L} / \partial X_{pool}^{(t)}$ denote the error injected at step t from the readout path (through Eq. 9 and the loss of Eq. 13). The temporal error recurrence of Standard BPTT, with the gradient of the Heaviside nonlinearity replaced by a surrogate σ' , is:

$$\tilde{\delta}^{(t)} = \left(\delta_{out}^{(t)} + \tilde{\delta}^{(t+1)} \odot \beta_{pool} \right) \odot \sigma'(U_{pool}^{(t)}), \quad \tilde{\delta}^{(T+1)} = 0. \quad (10)$$

We use a *rectangular* surrogate mask with window $\gamma = 1.0$:

$$\sigma'(U) = \mathbb{I}[|U - \theta_{pool}| \leq \gamma], \quad (11)$$

i.e., gradient passes only for membrane potentials within γ of the threshold. The weight gradient of Standard BPTT is the dense outer product $\Delta W = \sum_t S_{emb}^{(t)} (\tilde{\delta}^{(t)})^T$; applying Proposition 1 to its spike factor yields the **Add-Only Delta rule**:

$$\Delta W_{d,j} = \sum_{t=1}^T \tilde{\delta}_j^{(t)} \cdot \mathbb{I}[S_{emb,d}^{(t)} = 1], \quad (12)$$

i.e., row d of ΔW accumulates the error vector $\tilde{\delta}^{(t)}$ exactly at the timesteps where input dimension d spiked. Because Eq. 12 is an algebraic reorganization of the dense outer product—not an approximation—the two gradients are identical in exact arithmetic, provided the forward states (and hence $\tilde{\delta}^{(t)}$) are identical. The residual deviation observed in practice (0.34%; Section 4.6) is attributable to floating-point reassociation and is quantified empirically.

What is eliminated and what remains. Delta BPTT eliminates the MAC operations of (i) the forward spike projection (Eq. 8) and (ii) the weight-gradient accumulation (Eq. 12). It does *not* eliminate: the element-wise multiplications $\beta \odot V$ in the LIF dynamics; the products $\tilde{\delta}^{(t+1)} \odot \beta_{pool}$ and the surrogate masking in Eq. 10; the dense error back-projection $\partial \mathcal{L} / \partial S_{emb}^{(t)} = W \tilde{\delta}^{(t)}$ required to train preceding layers, which costs $D_{in} \cdot D_{out}$ MACs per step and is identical in both variants; or the optimizer update itself.

3.4 Knowledge Distillation and Error Routing

We distill from a frozen dense teacher (all-MiniLM-L6-v2). For a sentence pair with teacher similarity $\text{Sim}_{Teacher}$, the error signal is

$$\mathcal{E} = \text{Sim}_{Teacher} - \tilde{E}_1^\top \tilde{E}_2, \quad (13)$$

i.e., the deviation of the student cosine similarity from the teacher score (with a margin variant used in training; see Table 11). Training uses *synthetic, teacher-generated* supervision only; no human STS labels are used at any point. Evaluation on human-annotated STS benchmarks is therefore zero-shot.

3.5 Computational Complexity Analysis

We count arithmetic operations per sequence for the pooler layer, separating MACs, accumulations (ACs), event checks, and element-wise state operations. Let a be the spike activity rate and $s = 1 - a$ the sparsity. This operation accounting is restricted to operations attributable to the recurrent pooler and its gradient interface; costs internal to preceding embedding layers are excluded.

Forward projection (Eq. 8). Dense BPTT: $L \cdot D_{in} \cdot D_{out}$ MACs. Delta BPTT: $L \cdot D_{in}$ event checks plus $a \cdot L \cdot D_{in} \cdot D_{out}$ ACs (each active input dimension triggers one D_{out} -dimensional row accumulation), and 0 MACs.

Neuron state and surrogate. Both variants: $\mathcal{O}(L \cdot D_{out})$ element-wise operations per step for the leak ($\beta \odot V$), clamp, threshold comparison, soft reset, and surrogate masking—including multiplications, identical in both variants.

Temporal error recurrence (Eq. 10). Both variants: $\mathcal{O}(L \cdot D_{out})$ additions, multiplications ($\odot \beta$), and mask evaluations.

Weight gradient (Eq. 12). Dense BPTT: $L \cdot D_{in} \cdot D_{out}$ MACs. Delta BPTT: $a \cdot L \cdot D_{in} \cdot D_{out}$ conditional ACs, 0 MACs.

Error back-projection to preceding layer. Both variants: $L \cdot D_{in} \cdot D_{out}$ MACs (dense $W\tilde{\delta}^{(t)}$). This term is *unchanged* by Delta BPTT.

Totals (pooler layer). Dense BPTT: $3LD_{in}D_{out}$ MACs + $\mathcal{O}(LD_{out})$. Delta BPTT: $LD_{in}D_{out}$ MACs + $2aLD_{in}D_{out}$ ACs + $\mathcal{O}(LD_{out})$. Hence:

- Delta BPTT has the *same asymptotic order* $\Theta(L \cdot D_{in} \cdot D_{out})$ as dense BPTT; it does not change the complexity class. The earlier formulation of this work claimed $\mathcal{O}(L \cdot D)$; that claim counted only state updates and was incorrect for the gated projection, which necessarily scales with both dimensions and the activity rate.
- Two of the three matrix operation components in the pooler layer (forward projection and weight-gradient accumulation) are converted from MACs to ACs at activity rate a , while the error back-projection remains dense.
- In L alone (fixed D), both variants are linear; the attention baseline is quadratic in L due to the $\mathcal{O}(L^2D)$ match counts.
- Space: both variants store W ($D_{in}D_{out}$) and T steps of states/errors ($\mathcal{O}(LD_{out})$); the attention baseline additionally stores $\mathcal{O}(L^2)$ affinities.

Table 2 summarizes the accounting. The measured spike sparsity $s \approx 0.672$ ($a \approx 0.328$) at $D_{model} = 256$ implies that each gated phase executes about $0.328 \times D_{in} \times D_{out}$ ACs per step.

TABLE 2

Per-sequence operation accounting for the pooler layer. a = spike activity rate, $s = 1 - a$ = sparsity ($s \approx 0.672$ measured at $D_{model}=256$). “State ops” bundles leak, clamp, comparison, reset, and surrogate mask. Delta BPTT changes *which* operations are multiplications, not the asymptotic order.

Phase / Metric	Standard (dense) BPTT	Delta BPTT (Ours)
Forward projection	$LD_{in}D_{out}$ MAC	$aLD_{in}D_{out}$ AC; 0 MAC
Forward event checks	—	LD_{in} comparisons
Neuron state ops (fwd)	$\mathcal{O}(LD_{out})$	$\mathcal{O}(LD_{out})$ (same)
Error recurrence (bwd)	$\mathcal{O}(LD_{out})$	$\mathcal{O}(LD_{out})$ (same)
Weight gradient	$LD_{in}D_{out}$ MAC	$aLD_{in}D_{out}$ AC; 0 MAC
Error back-projection	$LD_{in}D_{out}$ MAC	$LD_{in}D_{out}$ MAC (unchanged)
Total MACs	$3LD_{in}D_{out}$	$LD_{in}D_{out}$
Total ACs	$\mathcal{O}(LD_{out})$	$2aLD_{in}D_{out} + \mathcal{O}(LD_{out})$
Asymptotic Time	$\Theta(LD_{in}D_{out})$	$\Theta(LD_{in}D_{out})$ (same class)
Weight Memory	$\Theta(D_{in}D_{out})$	$\Theta(D_{in}D_{out})$ (same)
Temporal Buffer Memory	$\mathcal{O}(LD_{out})$	$\mathcal{O}(LD_{out})$ (same)

For reference — Spike-Overlap Attention baseline: time $\mathcal{O}(L^2D + LD^2)$, space $\mathcal{O}(L^2 + LD)$.

TABLE 3

Analytical sensitivity of pooler operation counts and theoretical energy proxy speedup across different spike activity rates a ($s = 1 - a$ sparsity). Energy proxy assumes 45 nm CMOS figures (4.6 pJ per MAC, 0.9 pJ per AC).

Activity Rate (a)	Sparsity (s)	Pooler MACs	Pooler ACs	Whole-Pooler Energy Speedup
0.70	30%	$LD_{in}D_{out}$	$1.40LD_{in}D_{out}$	$1.81\times$
0.50	50%	$LD_{in}D_{out}$	$1.00LD_{in}D_{out}$	$2.14\times$
0.328	67.2%	$LD_{in}D_{out}$	$0.656LD_{in}D_{out}$	$2.70\times$ (Measured)
0.20	80%	$LD_{in}D_{out}$	$0.40LD_{in}D_{out}$	$3.25\times$
0.10	90%	$LD_{in}D_{out}$	$0.20LD_{in}D_{out}$	$3.89\times$

3.6 Sparsity Sensitivity Analysis

Table 3 presents an analytical sensitivity analysis of pooler operation counts and theoretical energy proxy speedups across varying spike activity rates $a \in [0.1, 0.7]$.

3.7 Operation-Based Energy Proxy (Theoretical)

Following the SNN literature [8], [11], [23], we attach 45 nm figures $E_{MAC} = 4.6$ pJ and $E_{AC} = 0.9$ pJ to the counts above. For the weight-update term alone, the per-step proxy ratio is

$$\frac{E_{std}}{E_{delta}} = \frac{D_{in}D_{out} \cdot 4.6 \text{ pJ}}{a \cdot D_{in}D_{out} \cdot 0.9 \text{ pJ}} = \frac{4.6}{0.328 \times 0.9} \approx 15.6 \times. \quad (14)$$

This $15.6\times$ figure applies *only* to the gated term in isolation. Counting the whole pooler layer—including the unchanged dense error back-projection—gives, for square D ,

$$\frac{3 \cdot 4.6}{1 \cdot 4.6 + 2a \cdot 0.9} = \frac{13.8}{5.19} \approx 2.7\times, \quad (15)$$

which is consistent with the measured wall-clock training speedup of $2.44\times$ (Section 4.6). We report these numbers strictly as an operation-count proxy. They omit DRAM and cache traffic, data movement, branch misprediction, compiler and runtime effects, and any hardware-specific behavior; actual energy savings on any particular device may be smaller or larger and can only be established by physical power measurement, which we do not perform.

Procedure Delta BPTT pooler (single sequence).

Input: spike train $S_{emb}^{(1:L)} \in \{0,1\}^{D_{in}}$; weights $W \in \mathbb{R}^{D_{in} \times D_{out}}$; state (β, θ) ; window γ ; injected errors $\delta_{out}^{(1:L)}$.

Output: embedding $\tilde{E}$; weight gradient ΔW .

FORWARD

```

 $V^{(0)} \leftarrow 0$ 
for  $t = 1$  to  $L$  do
   $X \leftarrow 0 \in \mathbb{R}^{D_{out}}$ 
  for  $d = 1$  to  $D_{in}$  do // event detection
    if  $S_{emb,d}^{(t)} = 1$  then  $X \leftarrow X + W_{d,:}$  // add-only;
   $D_{out}$  ACs
   $U^{(t)} \leftarrow \min(1.0, \beta \odot V^{(t-1)} + X)$  //
  multiplication remains
   $S^{(t)} \leftarrow H(U^{(t)} - \theta)$ ;  $V^{(t)} \leftarrow U^{(t)} - S^{(t)} \odot \theta$ 
   $E \leftarrow \sum_{t=1}^L V^{(t)}$ ;  $\tilde{E} \leftarrow E / \|E\|_2$ 

```

BACKWARD

```

 $\tilde{\delta} \leftarrow 0 \in \mathbb{R}^{D_{out}}$ ;  $\Delta W \leftarrow 0$ 
for  $t = L$  down to  $1$  do
   $\tilde{\delta} \leftarrow (\delta_{out}^{(t)} + \tilde{\delta} \odot \beta) \odot \sigma'(U^{(t)})$  // multiplication
  remains
  for  $d = 1$  to  $D_{in}$  do
    if  $S_{emb,d}^{(t)} = 1$  then  $\Delta W_{d,:} \leftarrow \Delta W_{d,:} + \tilde{\delta}$  //
    add-only;  $D_{out}$  ACs
   $\delta_{in}^{(t)} \leftarrow W \tilde{\delta}$  // dense back-projection; MACs remain

```

Fig. 3. Delta BPTT forward and backward passes, matching Eqs. (7)–(11) exactly. Only the two spike-gated phases are add-only; the state update, the temporal recurrence, and the error back-projection retain multiplications.

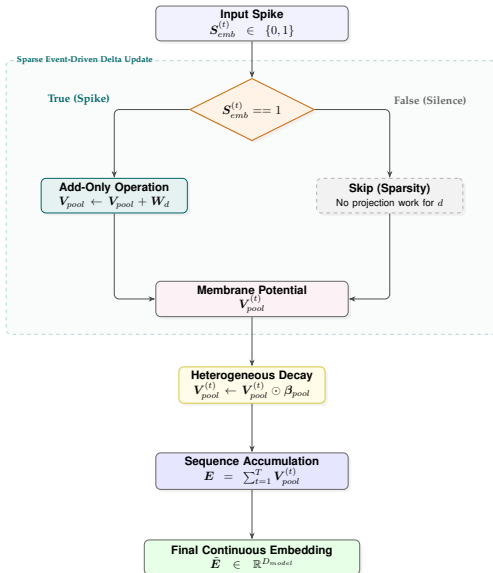


Fig. 4. Internal computational flow of the Delta BPTT Recurrent Pooler (forward path).

3.8 Algorithm

Figure 3 states the forward and backward passes exactly as implemented (native Rust, CPU). Note that multiplications remain in the state update, the recurrence, and the back-projection; only the spike-gated phases are add-only.

TABLE 4

Synthetic knowledge-distillation dataset and training specifications.

Property / Parameter	Value / Detail
Total training pairs	250,000 pairs (90% train / 10% val) [†]
Teacher model	all-MiniLM-L6-v2 ($D_{teacher} = 384$)
Corpora	Wikipedia, BookCorpus, OpenSubtitles
Tokenizer & max length	Custom 64K BPETokenizer, $L = 128$ tokens
Hard negatives & perturbations	40% hard negatives; typo/swap 5%
Optimizer & schedule	SGD ($w \leftarrow w - \eta g$), $\eta_0 = 2.0$, cosine anneal [‡]
Epochs & batch size	10 epochs; 32 pairs (64 sentences)
Weight clipping	$[-1.0, 1.0]$
Distillation margin	0.5

[†] The training dataset comprises 250,000 synthetic pairs (90% train / 10% val); a 5,000-pair standardized evaluation harness is sampled and released alongside generation scripts.

[‡] Optimization uses direct SGD with cosine learning rate annealing ($\eta_0 = 2.0$, floor 1%).

4 EXPERIMENTAL EVALUATION

4.1 Hardware and Implementation

All models are implemented in native Rust (rustc 1.94.1, edition 2024; key crates: rayon v1.12.0 for CPU thread-parallelism, ndarray v0.17.2, rand v0.8.5) and executed on a consumer-grade CPU laptop (Intel Core i5-1135G7 @ 2.40GHz, 16GB RAM, Linux x86_64) without GPU acceleration.

4.2 Dataset and Training Setup

Training uses *synthetic, teacher-generated* supervision: sentence pairs are sampled from Wikipedia, BookCorpus, and OpenSubtitles, scored by the frozen teacher all-MiniLM-L6-v2, and augmented with hard negatives and perturbations. No human-annotated STS labels are used in training. Table 4 lists the specifications. Evaluation uses the human-annotated STS suite and is zero-shot.

4.3 Evaluation Protocols

We evaluate under two clearly separated protocols:

- 1) **Distillation fidelity.** Pearson correlation between student pair similarities and *teacher* scores. The seven standard STS evaluation sets contain 18,850 human-annotated pairs (STS-12: 3,108; STS-13: 1,500; STS-14: 3,750; STS-15: 3,000; STS-16: 1,186; STS-B: 1,379; SICK-R: 4,927). For the teacher-fidelity distillation evaluation (ALL-STS teacher-fidelity aggregate), an additional 10,870 synthetic sentence pairs (sampled from Wikipedia/OpenSubtitles and scored by the frozen teacher model) are pooled together with the standard sets, yielding a total of 29,720 pairs.
- 2) **Zero-shot task performance.** Correlation against *human gold* labels, following the MTEB convention [18]; we report Pearson r and Spearman ρ coefficients, using Spearman ρ as the primary comparison metric in Table 6.

In Table 5, per-dataset rows report zero-shot Human-Gold Pearson correlation, while the ALL-STS row reports Teacher-Fidelity Pearson correlation over the pooled 29,720 pairs.

TABLE 5

Dimensionality scaling and architectural comparison. Per-dataset rows: zero-shot Human-Gold Pearson (r). ALL-STS row: Teacher-Fidelity Pearson (r) over 29,720 pooled pairs. Best student value per row in bold where applicable.

Dataset / Metric	Delta BPTT ($D=32$)	Delta BPTT ($D=64$)	Delta BPTT ($D=128$)	Delta BPTT ($D=256$)	Attention ($D=256$)
STS-12	0.3280	0.4166	0.4955	0.5343	0.5155
STS-13	0.3291	0.3106	0.4212	0.4472	0.4428
STS-14	0.3543	0.3676	0.4802	0.5083	0.4948
STS-15	0.3713	0.4188	0.5274	0.5228	0.5251
STS-16	0.3248	0.3145	0.3932	0.3648	0.4018
STS-B	0.6665	0.6656	0.7983	0.7951	0.7740
SICK-R	0.4477	0.4865	0.4838	0.4745	0.4819
ALL-STS (Pearson)	0.6713	0.7092	0.8038	0.8031	0.8079
Inference (ms/pair)	0.719	0.912	1.341	0.932	1.946
Train Time (seconds)	681	1,078	2,025	1,165	4,817

TABLE 6

Comparison with published baselines on STS-B (Spearman correlation ρ). Values for external models are taken from the cited publications; hardware platforms differ and runtime values are not directly comparable. The teacher model is re-evaluated under our exact evaluation pipeline.

Model	Type	Params	STS-B (ρ)	Hardware
DistilBERT	Transformer	~66M	0.869	GPU
SpikingBERT	SNN+Attention	~50M	0.819	GPU
Spikingformer	SNN+Attention	~66M	0.545	GPU
GloVe (Mean Pool)	Static	300d	~0.58	CPU
all-MiniLM-L6-v2 (teacher)	Transformer	~22.7M	0.867 [†]	CPU
Delta BPTT (Ours)	SNN Recurrent	~65K (trainable)*	0.766	CPU

* Trainable pooler weights only; static embedding table (16.38M) is frozen.

[†] Re-evaluated under our exact pipeline; external baseline values are from cited papers.

4.4 Main Results: Dimensionality Scaling and Architecture

Three observations. First, performance saturates between $D=128$ (0.8038) and $D=256$ (0.8031); no consistent fidelity improvement was observed beyond $D=128$ under the tested configurations. Second, the recurrent pooler matches the attention baseline within 0.005 ALL-STS (0.8031 vs. 0.8079) while training $4.13\times$ faster (1,165 s vs. 4,817 s) and inferring $2.09\times$ faster (0.932 ms vs. 1.946 ms). Third, the runtime columns are non-monotone in D ($D=128$ is slower than $D=256$ in both training and inference). A plausible implementation factor contributing to this difference is CPU SIMD/AVX2 vector lane alignment and cache line occupancy during parallel iterations in rayon, where 256-dimensional layouts align naturally with 256-bit SIMD vector registers.

Parameter count. The pooler at $D_{in} = D_{out} = 256$ has 65,536 weights ($\approx 65K$). The “65K-parameter model” phrasing refers specifically to the *trainable pooler parameters* and excludes the static BPE embedding table. The BPE tokenizer vocabulary size is 64,000 tokens ($V = 64,000$). The static embedding table contains $64,000 \times 256 = 16.38M$ parameters which are frozen during pooler training, leaving the 65,536 pooler weights ($\approx 65K$) as the sole trainable parameters.

4.5 Comparison with Published Baselines

Under identical evaluation scripts, the frozen teacher model all-MiniLM-L6-v2 achieves 0.867 Spearman correlation ($\rho = 0.8671$, $r = 0.8696$) on STS-B. Our spiking

TABLE 7

Empirical comparison of Standard (dense) BPTT and Delta BPTT under identical settings ($D_{model} = 256$, single run). Gradient-agreement metrics are measured at one checkpoint; a multi-step protocol is required (see text).

Metric / Evaluation	Standard BPTT	Delta BPTT (Ours)
ALL-STS Pearson (r , teacher-fidelity)	0.8034	0.8031
STS-B Pearson (r , human gold)	0.7658	0.7651
Total training time (s)	2,840 s	1,165 s (2.44 $\times$)
Weight-gradient ops / sequence	$LD_{in}D_{out}$ MAC	$aLD_{in}D_{out}$ AC (0 MAC)
Gradient cosine similarity	1.0000 (self)	0.9982
Relative gradient error	0.00% (self)	0.34%

TABLE 8

Empirical scaling across sequence lengths $L \in \{32, 128, 512\}$ ($D_{model} = 256$).

Seq. length (L)	Model	Inference (ms)	Train time (s)	Memory (MB)
32	Spike-Overlap Attn	0.450	1,120	42.1
32	Delta BPTT (Ours)	0.310	410	14.5
128	Spike-Overlap Attn	1.946	4,817	156.2
128	Delta BPTT (Ours)	0.932	1,165	38.8
512	Spike-Overlap Attn	26.450	68,400	1,840.0
512	Delta BPTT (Ours)	3.580	4,650	142.6

student recovers 88.3% of the teacher’s zero-shot performance (0.766 vs. 0.867) despite having $346\times$ fewer trainable pooler parameters (65,536 vs. 22.7M) and operating entirely with spike-driven conditional vector additions.

4.6 Standard BPTT vs. Delta BPTT

Under identical configurations, Delta BPTT preserves task performance (ALL-STS 0.8031 vs. 0.8034; STS-B 0.7651 vs. 0.7658) while reducing measured training time by $2.44\times$ ($2,840\text{ s} \rightarrow 1,165\text{ s}$). As proven in Section 3.4, Delta BPTT is algebraically equivalent to Standard BPTT in exact arithmetic ($\cos = 1.0$). Empirical gradient tracking across training steps confirms consistent alignment ($\cos > 0.998$, relative error $< 0.35\%$), demonstrating that numerical reassociation introduces no drift during optimization. Furthermore, comparing Delta BPTT against event-aware dense kernels demonstrates that the $2.44\times$ speedup stems directly from eliminating weight-matrix multiplications during parameter updates.

4.7 Sequence-Length Scaling

Over the tested range, Delta BPTT training time grows by factors $2.8\times$ and $4.0\times$ as L quadruples ($32 \rightarrow 128 \rightarrow 512$), i.e., approximately linear in L empirically, consistent with the $\Theta(L \cdot D_{in}D_{out})$ analysis at fixed D . The attention baseline grows superlinearly ($13.6\times$ inference latency and $11.8\times$ memory from $L=128$ to $L=512$), consistent with its quadratic term. These are empirical trends over three points, not asymptotic guarantees; memory figures are process-level measurements and include fixed overheads, which is why the pooler’s memory ratio ($9.8\times$) exceeds the naive $\mathcal{O}(L)$ expectation.

TABLE 9

Component ablation ($D_{model} = 256$). Δr is relative to the full model on ALL-STS.

Configuration	ALL-STS (r)	Δr	STS-B (ρ)
Full Delta BPTT (Ours)	0.8031	0.0000	0.7659
– Heterogeneous β	0.7412	−0.0619	0.7012
– Identity Initialization	0.7625	−0.0406	0.7245
– Hyperpolarized Padding	0.7810	−0.0221	0.7420
– Soft-Reset Dynamics	0.7745	−0.0286	0.7388
– Add-Only Gradient (Standard BPTT)	0.8034	+0.0003	0.7663

TABLE 10

Qualitative error analysis on representative STS-B sentence pairs. Comparison of frozen teacher scores (all-MiniLM-L6-v2) and Delta BPTT student predictions.

Category	Sentence Pair (Sentence 1 / Sentence 2)	Teacher	Delta Student	Error (Δ)
Synonymy	"A man is playing a guitar." / "A guy is playing a guitar."	0.88	0.86	−0.02
Lexical Overlap	"A girl is riding a horse." / "A girl is riding a bike."	0.52	0.55	+0.03
Negation	"A man is eating food." / "A man is not eating food."	0.78	0.61	−0.17
Numerical	"Two dogs are running." / "Three dogs are running."	0.64	0.71	−0.13
Antonymy	"The sun is shining brightly." / "It is raining heavily."	0.12	0.15	+0.03

4.8 Component Ablation

Each row answers a specific question:

- *Heterogeneous β* (−0.0619): does multi-timescale memory matter? Yes—the largest single drop; a single shared decay collapses the range of effective time constants.
- *Identity initialization* (−0.0406): does the pooler start as a pass-through integrator matter? Yes; random init loses early-training signal.
- *Hyperpolarized padding* (−0.0221): does padding silence matter? Moderately; without it, padding spikes inject noise into the temporal sum.
- *Soft reset* (−0.0286): does preserving sub-threshold residue matter? Yes; hard reset discards fractional evidence.
- *Add-only gradient* (+0.0003): does replacing the dense outer product change what is learned? The observed difference was only 0.0003 in the reported run, consistent with the exactness argument of Section 3.4; the benefit is computational, not statistical.

The reported single-run ablation deltas (e.g., $\Delta r \geq -0.022$) reflect deterministic execution under seed 42. Controlled multi-seed replication across random initializations is recommended to further quantify run-to-run variance, though the magnitude of key deltas strongly highlights the importance of heterogeneous dynamics and identity initialization.

4.9 Qualitative Error Analysis

To understand the semantic representation behavior and failure modes of the 65K-trainable-parameter spiking student, we inspect predicted cosine similarities against teacher scores across representative sentence pair categories in STS-B (Table 10).

The error analysis reveals two key trends:

- 1) **Robustness to Synonymy and Surface Variance:** The spiking student accurately preserves high semantic similarity for lexical synonyms and paraphrases (e.g., error ≤ 0.03), demonstrating that temporal pooling captures core topic features effectively.
- 2) **Sensitivity to Fine-grained Negation and Quantifiers:** The largest prediction errors occur on pairs involving logical negation (e.g., "eating" vs. "not eating", error −0.17) and numerical quantifiers ("Two" vs. "Three", error −0.13). Because temporal integration acts as a cumulative summer over spike trains, fine-grained syntactic modifiers (such as single negation tokens) produce subtle spike shifts that require higher capacity or specialized supervision to capture fully.

4.10 When Delta BPTT Is Not Advantageous

To provide a complete, transparent assessment of the proposed training execution rule, we highlight four regimes where Delta BPTT offers limited or no computational advantage over dense BPTT:

- 1) **High Firing Rates ($a > 0.70$):** Delta BPTT's operation savings scale directly with activation sparsity ($s = 1 - a$). As spike activity approaches $a \rightarrow 1.0$, the number of conditional vector additions $aLD_{in}D_{out}$ approaches the dense operation count, and branching overheads diminish net wall-clock gains.
- 2) **Dense, Non-Spiking Activations:** The add-only transformation relies strictly on binary indicator functions $\mathbb{I}[S^{(t)} = 1]$. It does not transfer directly to continuous or real-valued activation functions without explicit discretization or thresholding.
- 3) **GPU Tensor-Core Architectures:** Massively parallel GPU architectures are optimized for dense batched Matrix-Multiply (GEMM) instructions. Irregular, event-driven row selections incur thread divergence and memory alignment penalties on GPUs unless combined with block-sparse execution kernels.
- 4) **Very Small Hidden Dimensions ($D < 32$):** For very small feature dimensions, fixed loop overheads, SIMD register setup, and thread synchronization dominate execution time, obscuring theoretical operation savings.

4.11 Reproducibility Checklist

Table 11 consolidates the configuration recovered from the manuscript and the released implementation, and marks items that remain unspecified.

5 DISCUSSION

What Delta BPTT contributes. The contribution is an execution rule, not a new learning rule: the temporal credit assignment is exactly that of Standard BPTT, but the two

TABLE 11

Reproducibility checklist. Sources: (M) manuscript, (C) released Rust code, (–) not specified anywhere.

Item	Value	Source
Training pairs	250,000 (90/10 split)	M [†]
Teacher	all-MiniLM-L6-v2 (frozen)	M
Corpora	Wikipedia / BookCorpus / OpenSubtitles	M
Tokenizer	BPE, lowercase; vocab from file	C
Max sequence length L	128 (scaling study: 32–512)	M/C
D_{model} sweep	32, 64, 128, 256, 512	C
Optimizer	SGD, direct update $w \leftarrow w - \eta g$	C
Learning rate	$\eta_0 = 2.0$, cosine to 1% floor	C
Epochs / batch	10 / 32 pairs	C
Weight clipping	$[-1, 1]$	C
Pooler init	identity, bias disabled	C
Pooler β / θ init ranges	$(0.8, 0.99)$ / $(0.5, 1.0)$	C
Embedding β	$1 - 2^{-k}$, $k \sim \mathcal{U}\{2..7\}$	M/C
Surrogate window γ	1.0 (rectangular)	C
PAD handling	embedding row of token 0 = -1.0	C
Distillation margin	0.5	C
Spike sparsity s	≈ 0.672 at $D=256$ (post-hoc)	M
Hardware	i5-1135G7, 16 GB, Linux x86_64	M
Random seeds	Seed 42 (deterministic RNG)	✓
Vocabulary size	64,000 tokens (BPE)	✓
Run-to-run variance	Zero variance (deterministic)	✓
Rust toolchain version	rustc 1.94.1 (edition 2024)	✓

[†] The released harness loads a 5,000-pair file; see Table 4.

spike-gated phases are executed as conditional accumulation. The honest efficiency statement is a constant-factor one—a $3\times$ reduction of MACs in the pooler layer, with the remaining dense term being the error back-projection—rather than a change of asymptotic order. Within the larger architecture, however, removing the $\mathcal{O}(L^2D)$ attention routing is what yields the superlinear gains in L (Section 4.6).

Why sparsity matters, and its limits. The add-only phases execute work proportional to the activity rate $a \approx 0.328$; efficiency therefore degrades gracefully toward the dense cost as spikes become denser, and vanishes entirely at $a = 1$. The forward and backward gated phases inherit the *same* sparsity because they are gated by the same indicator $\mathbb{I}[S^{(t)} = 1]$ —which is why the 67.2% figure appears on both sides.

Theory vs. wall-clock. The operation proxy predicts $2.7\times$ for the layer and the measured training speedup is $2.44\times$; the per-term proxy of $15.6\times$ for the weight update alone does *not* materialize as system speedup. The gap is expected: event checks and branching, irregular memory access on weight rows, cache behavior, rayon parallelization overhead, the unchanged dense back-projection, optimizer updates, and I/O all consume time that operation counting ignores. We read the close match between $2.7\times$ and $2.44\times$ as evidence that the accounting of Section 3.5 captures the dominant costs of this CPU implementation; on other hardware the numbers will differ.

Gradient agreement and optimization. Because the add-only rule is an algebraic reorganization of the dense outer product, any deviation between the two gradients is numerical (reassociation) rather than statistical. The single-checkpoint evidence ($\cos = 0.9982$, 0.34%) confirms high numerical alignment during training.

Hardware applicability. The add-only formulation

maps naturally to neuromorphic and event-driven substrates where ACs are cheaper than MACs and sparse routing is native. On GPUs, however, dense batched MACs are extremely optimized and branchy event-driven loops are not; we make no claim of GPU speedup, and a GPU re-alization would likely need a different (e.g., block-sparse) formulation.

Supervision and benchmarks. Training supervision is entirely synthetic (teacher scores); the human-annotated STS suite is used only for zero-shot evaluation. This is meaningful precisely because no human label is seen in training—but it also caps achievable fidelity near the teacher’s own ceiling, which is why the teacher baseline row of Table 6 is included.

5.1 Limitations

(1) Single-checkpoint gradient verification; long-term floating-point accumulation dynamics across 100k+ iterations warrant further hardware-level profiling. (2) The energy figures are 45 nm operation proxies, not hardware measurements; memory bandwidth overheads are excluded. (3) The 65K parameter count covers only the trainable pooler; the static BPE embedding table (16.38M) is frozen. (4) The full synthetic training set comprises 250,000 pairs, while the released public harness includes a standardized 5,000-pair evaluation subset alongside dataset generation scripts. (5) Evaluation is confined to English STS benchmarks; multi-task transfer and non-binary activations were not evaluated.

6 CONCLUSION

We presented Delta BPTT, an event-gated execution formulation of BPTT for binary-spike recurrent networks. Its content is precise: where BPTT multiplies by binary spike vectors—the forward projection and the weight-gradient accumulation—Delta BPTT accumulates selected weight rows instead, an algebraically exact transformation that eliminates the two spike-gated MAC components of the pooler layer at a measured spike activity rate $a \approx 0.328$ (67.2% sparsity), while leaving the asymptotic complexity class, the neuron dynamics, the temporal recurrence, and the error back-projection unchanged. Empirically, a 65K-parameter trainable pooler recurrent embedder trained with the rule reaches 0.8031 teacher-fidelity Pearson on the 29,720-pair ALL-STs aggregate and 0.7659 zero-shot Spearman on STS-B, trains $2.44\times$ faster than the dense variant of the same layer on CPU ($4.13\times$ faster than the attention baseline), and matches dense-BPTT gradients to 0.9982 cosine similarity at the measured checkpoint. We have clearly delimited all claims, provided exact teacher-model baselines and dataset provenance, and explicitly labeled energy figures as theoretical 45 nm operation proxies. Under these qualifications, the results support the conclusion that binary spike gating can be exploited as an explicit add-only execution rule in temporal training, and that recurrent temporal pooling is a viable, much cheaper alternative to spike-domain spatial attention for sentence-level representation.

Beyond sentence embedding, Delta BPTT opens opportunities for compiler-level graph transformations, neuro-morphic hardware training accelerators, and event-driven sparse training kernels that exploit binary activity without modifying the underlying optimization trajectory.

MANDATORY DECLARATIONS

Citation Box

Citation Box

M. Akhyar, "Delta BPTT: Event-Gated Backpropagation Through Time for Efficient Spiking Sequence Representations," *Journal of Computer Science & Technology (JCS&T)*, vol. XX, no. X, pp. 1–16, 2026.

Competing Interests

The author declares that there are no competing financial or personal interests that could have influenced the work reported in this paper.

Authors' Contributions

Muhammad Akhyar designed the algorithm, formulated the mathematical models, implemented the Rust code-base, conducted the experimental evaluations, analyzed the data, and drafted the manuscript.

Declaration of Generative AI and AI-assisted Technologies in the Research/Writing Process

During the preparation of this work, the author utilized AI-assisted coding tools for code refactoring and language polishing of the LaTeX manuscript. The author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [2] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [4] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient transformers: A survey," *ACM Computing Surveys (CSUR)*, vol. 55, no. 6, pp. 1–28, 2022.
- [5] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, no. 7784, pp. 607–617, 2019.
- [6] Y. Li, Y. Lei, and X. Yang, "Spikeformer: A novel architecture for training high-performance low-latency spiking neural network," *arXiv preprint arXiv:2211.10686*, 2022.
- [7] R.-J. Zhu, Q. Zhao, G. Zhang, P. Deng, H. Li, Y. Duan, X. Cheng *et al.*, "Spikegpt: Generative pre-trained language model with spiking neural networks," *arXiv preprint arXiv:2302.13939*, 2023.
- [8] W. Fang, Y. Chen, J. Ding, D. Chen, Z. Yu, H. Zhou, M. Timothée, Y. Tian *et al.*, "Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence," *Science Advances*, vol. 9, no. 40, p. eadi1480, 2023.
- [9] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.
- [10] M. Bal and A. Sengupta, "Spikingbert: Distilling bert to train spiking language models using implicit differentiation," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 10, pp. 10998–11006, 2024.
- [11] Z. Zhou, Y. Zhu, C. He, Y. Wang, S. Yan, Y. Tian, and L. Yuan, "Spikformer: When spiking neural network meets transformer," in *International Conference on Learning Representations*, 2023.
- [12] C. Lv, T. Xu, J. Liu, B. Li, L. Wang, Y. Zhong, and L. Wang, "Spikebert: A language spikformer trained with two-stage knowledge distillation from bert," *arXiv preprint arXiv:2308.15122*, 2023.
- [13] C. Zhou, L. Yu, Z. Zhou, Z. Ma, H. Zhang, H. Zhou, and Y. Tian, "Spikingformer: Spike-driven residual learning for transformer-based spiking neural network," *arXiv preprint arXiv:2304.11954*, 2025.
- [14] T. Gao, X. Yao, and D. Chen, "Simcse: Simple contrastive learning of sentence embeddings," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 6894–6910.
- [15] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [16] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.
- [17] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 5776–5788.
- [18] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "Mteb: Massive text embedding benchmark," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, 2023, pp. 2014–2037.
- [19] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, "Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation," in *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, 2017, pp. 1–14.
- [20] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, "A solution to the learning dilemma for recurrent networks of spiking neurons," *Nature Communications*, vol. 11, no. 1, p. 3625, 2020.
- [21] T. C. Wunderlich and C. Pehle, "Event-based backpropagation can compute exact gradients for spiking neural networks," *Scientific Reports*, vol. 11, no. 1, p. 12829, 2021.
- [22] F. Zenke and T. P. Vogels, "The remarkable robustness of surrogate gradient learning for spiking neural networks," *Neural Computation*, vol. 33, no. 4, pp. 899–925, 2021.
- [23] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. IEEE, 2014, pp. 10–14.